Documentación Técnica - Eco Flow Simulation

Tabla de Contenidos

1. Visión General
2. Análisis del Sistema
3. Arquitectura y Procesos
4. Secciones Críticas
5. Recursos Críticos
6. Escenarios de Concurrencia
7. Utilidad del Proyecto
8. Aprendizajes y Competencias Desarrolladas

Asunciones del Sistema

1. Escalamiento Temporal y Ciclo de Vida

Tiempo Simulado vs. Real: Dado que el sistema opera activamente de 06:00 a 18:00 (12 horas) durante 30 días, se asume un factor de escalamiento temporal en el Orquestador para viabilizar la ejecución del mes de simulación en un tiempo de cómputo práctico, mapeando horas simuladas a fracciones de segundo reales.

Persistencia de Solicitudes: Las peticiones que no logran obtener reserva en un nodo durante su bloque horario no son descartadas, sino que se asumen como solicitudes en estado APLAZADA y se transfieren a la cola de contención de la hora siguiente mediante la función de recuperación.

2. Distribución de Carga y Probabilidades

Comportamiento Estocástico: El volumen máximo de 250 solicitudes diarias se asume distribuido a lo largo de las 12 horas activas siguiendo una distribución de Poisson, dividiendo la carga generada de forma concurrente entre el proceso Residencial y el Industrial.

Distribución de Acciones: El sistema asume que la probabilidad del 50% de realizar una reserva es absoluta. El 50% de probabilidad restante se asume distribuido entre las otras cuatro acciones del autómata de estado del usuario (esperar asignación, consumir/trabajar, cancelar solicitud, consultar presión, pagar excedente).

3. Lógica de Negocio y Auditoría

Conversión Volumétrica: El umbral de alerta del Auditor está definido en litros (>500 L por hora), mientras que los reportes globales exigen metros cúbicos. Se asume una conversión interna constante (1 m^3 = 1000 L) en el momento de actualizar las métricas en la memoria compartida.

Validación Asíncrona: Se asume que la validación de un consumo crítico (>500 L) por parte del Auditor de Flujo es un proceso no bloqueante para el recurso hídrico. El nodo se libera al terminar la hora de consumo, independientemente del tiempo que tarde el Auditor en clasificar la señal como "crítica" o "estándar".

Manejo de Amonestaciones: Se asume que cuando un usuario intenta cancelar una reserva sin tener un bloqueo activo, el sistema actualiza el contador de amonestaciones_digitales y aborta o reinicia la transacción de ese usuario sin bloquear el acceso a la lectura del nodo para los demás.

4. Arquitectura de Usuarios

Mapeo de Usuarios a Hilos: Aunque el sistema general usa una arquitectura de procesos pesados (Orquestador, Auditor, Nodos Principales), se asume que las hasta 250 "solicitudes de servicio" diarias se manejan internamente como hilos (threads) dentro de los procesos Residencial e Industrial (informacion_hilos). Levantar 250 procesos pesados diarios a nivel de sistema operativo introduciría un overhead de cambio de contexto injustificado.

5. Parámetros de Población y Generación de Carga

Población y Asignación: El sistema opera con M = 18 usuarios residenciales y N = 10 usuarios industriales. La asignación de estos usuarios a las solicitudes generadas sigue una distribución uniforme.

Llegada de Solicitudes: El número de clientes que llega en una hora particular sigue una distribución de Poisson. Si en un momento dado se excede el límite máximo de solicitudes diarias (250), se aplica un truncamiento.

Asincronía en Generación (Hilos): Los procesos instancian todos los hilos (solicitudes) correspondientes a un bloque horario en el mismo instante lógico. Para simular el envío asíncrono a lo largo del bloque, cada hilo espera individualmente un tiempo basado en una distribución exponencial con media de 20ms antes de intentar su acción.

Límites de Creación: Los procesos residenciales e industriales generan hilos hasta alcanzar sus límites máximos de capacidad por bloque (Nota: Las cotas exactas de creación y variables de relación matemática deben definirse explícitamente en la implementación, ya que los valores de las ecuaciones originales no se especifican).

6. Modelo de Tiempo Discreto y Eventos Programados

Time Slots Estrictos: El día se divide en 12 bloques exactos (ej. 06:00-07:00). Las reservas están acotadas a la duración restante del bloque. Si un hilo obtiene el nodo a las 06:15, su tiempo máximo de consumo será de los 45 minutos lógicos restantes.

Eventos de Sincronización Temporal:

Monitor del Sistema: Despierta exactamente a la mitad de cada bloque horario (minuto 30) para realizar y registrar las lecturas de presión y ocupación.

Auditor de Flujo: Despierta obligatoriamente al finalizar cada hora lógica (minuto 60) para procesar el cálculo total de los metros cúbicos consumidos en ese bloque.

Manejo de Sobrecarga y Rezagados: La acumulación en colas (ej. 125 demandas para 100 espacios) es un cuello de botella deliberado. A las 18:00 (fin del ciclo), el hilo principal ejecuta un broadcast para despertar a todos los hilos bloqueados, registrar su tiempo de espera como fallido ("no atendidos") y forzar una terminación limpia.

7. Máquina de Estados y Exclusividad de Recursos

Flujo Lógico de Hilos:

Nacimiento: Creación y espera asíncrona (delay exponencial).

Selección Primaria: 50% de probabilidad estricta para solicitar reserva. El 50% restante se distribuye entre consultar y cancelar.

Ejecución:

Consulta: Lee el estado del sistema mediante patrón Lectores/Escritores y termina.

Cancelación (Escritor): Acción estocástica concurrente. Puede ejecutarse sin estado previo. Si el ID no posee reservas, emite una amonestación digital y termina. Si posee reserva o está en cola, libera el espacio y termina.

Reserva: Si no hay nodos libres, el hilo se bloquea en la estructura de espera.

Consumo: Posterior a la asignación, se elige entre: Consumo estándar, Consumo crítico (> 500L, despierta al auditor) o Liberación anticipada (pagar excedente). El estándar y el crítico son mutuamente excluyentes.

Finalización: Libera la válvula y notifica validaciones si aplica.

Unicidad de Estado (Transacción Atómica): Un usuario (ID) no puede ocupar más de una válvula principal ni tener entradas duplicadas en la cola de espera. La operación de reserva requiere:

Validación: Buscar el ID en nodos activos y en la cola antes de pedir el semáforo.

Resolución: Si el ID ya existe, la solicitud es una anomalía concurrente, se rechaza y el hilo termina.

8. Casos Límite y Colisiones

Liberación Anticipada (Pagar Excedente): Ocurre como decisión exclusiva dentro de la sección crítica (estado "consumir"). Libera el nodo instantáneamente, entregándolo al primer hilo en la cola de espera de ese bloque.

Colisión de Tiempos: El cálculo del tiempo aleatorio para una liberación anticipada está estrictamente acotado por el tiempo lógico restante del bloque.

Desperdicio de Señales de Despertar: Si se ejecuta una liberación anticipada fracciones de milisegundo antes de terminar la hora, el primer hilo en cola es despertado y adquiere el recurso, pero será finalizado inmediatamente o purgado por el evento de cambio de hora (Time Slot estricto).

Visión General

Eco Flow Simulation es un sistema distribuido de simulación de gestión de recursos hídricos que modela el consumo de agua en un entorno urbano con múltiples tipos de usuarios (residenciales e industriales). El sistema implementa patrones avanzados de concurrencia y sincronización entre procesos mediante memoria compartida y múltiples mecanismos de IPC (Inter-Process Communication).

Objetivo Principal

Simular un sistema de distribución de agua donde múltiples usuarios compiten por recursos limitados (10 nodos de flujo) durante un período de tiempo determinado (30 días, 12 horas por día), registrando métricas de consumo, eficiencia y patrones de uso.

Análisis del Sistema

Componentes Principales

1. Orquestador (eco_flow_main.c)

- **Responsabilidad**: Coordinación central de toda la simulación
- **Funciones clave**:

- Creación y gestión de memoria compartida

- Lanzamiento y sincronización de procesos hijos

- Control del flujo temporal (días, horas, minutos)

- Recopilación de estadísticas finales

2. Procesos de Consumo

- **Nodo Residencial** (`residencial.c`): Gestiona solicitudes de usuarios residenciales
- **Nodo Industrial** (`industrial.c`): Gestiona solicitudes de usuarios industriales
- Ambos implementan patrones de generación de solicitudes basados en distribuciones de Poisson

3. Sistema de Auditoría (`auditor.c`)

- Monitorea consumos críticos (>500 litros/hora)
- Genera alertas y amonestaciones
- Calcula estadísticas horarias del sistema

4. Sistema de Monitoreo (`monitor.c`)

- Realiza lecturas de presión del sistema
- Registra estados de ocupación de nodos
- Calcula métricas de eficiencia

5. Sistema de Visualización (`monitor_nodos.c`)

- Interfaz en tiempo real para visualización del estado del sistema
- Muestra información detallada de cada nodo

Flujo de Operación

```
1. Orquestador crea memoria compartida

2. Lanza procesos hijos (residencial, industrial, auditor, monitor)

3. Inicia ciclo de simulación (días → horas → minutos)

4. Procesos generan solicitudes concurrentemente

5. Auditor procesa consumos críticos

6. Monitor registra estados del sistema

7. Orquestador recopila estadísticas finales
```

Arquitectura y Procesos

Estructura de Procesos

El sistema implementa una arquitectura de **procesos pesados** con comunicación mediante memoria compartida:

```
Orquestador (PID principal)

├── Proceso Residencial (hijos)

├── Proceso Industrial (hijos)

├── Proceso Auditor (hijos)

└── Proceso Monitor (hijos)
```

Memoria Compartida

Estructura principal (`MemoriaCompartida` en `ipc_shared.h`):

```
typedef struct {

    NodoFlujo valvulas[NUM_NODOS];        // 10 nodos de flujo

    double total_metros_cubicos;          // Métricas globales

    int amonestaciones_digitales;

    int senales_criticas;

    int senales_estandar;
```

```
    // Estado de simulación
    int dia_actual, hora_actual, minuto_actual;
    bool simulacion_activa;
    // Sincronización
    pthread_mutex_t mutex_metricas;
    pthread_rwlock_t mutex_nodos;
    sem_t sem_nodo_industrial, sem_nodo_residencial;
    sem_t sem_auditor, sem_monitoreo;
    // ... más semáforos
} MemoriaCompartida;
```

Mecanismos de Sincronización

1. **Semáforos**: Coordinación entre procesos
2. **Mutex**: Protección de secciones críticas
3. **RWLocks**: Acceso concurrente de lectores/escritores
4. **Señales**: Comunicación de terminación

Secciones Críticas

1. Gestión de Nodos de Flujo

```
// Sección crítica: Reserva de nodos
pthread_rwlock_wrlock(&shm->valvulas[nodo].rwlock_nodo);
if (shm->valvulas[nodo].ocupado) {
    // Nodo no disponible
    pthread_rwlock_unlock(&shm->valvulas[nodo].rwlock_nodo);
    return -1;
}
shm->valvulas[nodo].ocupado = true;
shm->valvulas[nodo].usuario_id = usuario_id;
pthread_rwlock_unlock(&shm->valvulas[nodo].rwlock_nodo);
```

2. Actualización de Métricas Globales

```
// Sección crítica: Actualización de estadísticas
pthread_mutex_lock(&shm->mutex_metricas);
shm->total_metros_cubicos += consumo_horario;
shm->amonestaciones_digitales++;
shm->tiempo_espera_total_micros += tiempo_espera;
pthread_mutex_unlock(&shm->mutex_metricas);
```

3. Detección de Consumos Críticos

```
// Sección crítica: Registro de consumo crítico
if (consumo > LIMITE_CONSUMO_CRITICO) {
    pthread_mutex_lock(&shm->mutex_consumo_critico);
    shm->nodo_consumo_critico_id = nodo_id;
    pthread_mutex_unlock(&shm->mutex_consumo_critico);
    sem_post(&shm->sem_auditor_listas);
}
```

4. Control de Concurrencia de Solicitudes

```
// Sección crítica: Gestión de nodos libres
sem_wait(&shm->sem_nodos_libres);
int nodo = obtener_nodo_disponible();
if (nodo == -1) {
    sem_post(&shm->sem_nodos_libres);
    return -1;
}
// ... usar nodo ...
sem_post(&shm->sem_nodos_libres);
```

Recursos Críticos

1. Memoria Compartida

- **Descripción**: Área de memoria compartida entre todos los procesos
- **Tamaño**: Estructura `MemoriaCompartida` (~2KB)
- **Protección**: Múltiples mecanismos de sincronización
- **Riesgos**: Race conditions, deadlocks, inconsistencia de datos

2. Nodos de Flujo (10 válvulas)

- **Descripción**: Recursos limitados de acceso exclusivo
- **Capacidad**: Máximo 10 usuarios concurrentes
- **Gestión**: Reader-writer locks por nodo
- **Contención**: Punto principal de competencia entre procesos

3. Semáforos de Sincronización

- **Descripción**: Coordinación temporal entre procesos
- **Tipos**: 15+ semáforos diferentes
- **Estados**: Binarios y contadores
- **Riesgos**: Deadlocks si no se manejan correctamente

4. Métricas Globales

- **Descripción**: Acumuladores de estadísticas del sistema
- **Protección**: Mutex exclusivo
- **Actualización**: Frecuente y concurrente
- **Consistencia**: Crítica para resultados finales

Escenarios de Concurrencia

1. Competencia por Nodos

Escenario: Múltiples procesos intentan reservar el mismo nodo simultáneamente

Solución Implementada:

```
// Reader-writer lock por nodo
pthread_rwlock_wrlock(&shm->valvulas[nodo].rwlock_nodo);
if (!shm->valvulas[nodo].ocupado) {
    shm->valvulas[nodo].ocupado = true;
    shm->valvulas[nodo].usuario_id = usuario_id;
    success = true;
}
pthread_rwlock_unlock(&shm->valvulas[nodo].rwlock_nodo);
```

2. Actualización Concurrente de Métricas

Escenario: Varios procesos actualizan estadísticas simultáneamente

Solución Implementada:

```
// Mutex global para métricas
pthread_mutex_lock(&shm->mutex_metricas);
shm->total_metros_cubicos += consumo;
shm->total_consultas_realizadas++;
pthread_mutex_unlock(&shm->mutex_metricas);
```

3. Sincronización Temporal

Escenario: Coordinación del avance del tiempo simulado

Solución Implementada:

```
// Semáforos para control horario
sem_wait(&shm->sem_hora_empezada_residencial);
// ... procesar hora actual ...
sem_post(&shm->sem_nodo_residencial_listo_hora);
```

4. Detección de Consumos Críticos

Escenario: Notificación asíncrona de consumos excesivos

Solución Implementada:

```
// Notificación al auditor
pthread_mutex_lock(&shm->mutex_consumo_critico);
shm->nodo_consumo_critico_id = nodo_id;
pthread_mutex_unlock(&shm->mutex_consumo_critico);
sem_post(&shm->sem_auditor_listas);
```

5. Gestión de Solicitudes Aplazadas

Escenario: Solicitudes que deben transferirse entre horas

Solución Implementada:

```
// Transferencia atómica de solicitudes
void recuperar_solicitudes_aplazadas(int *recuperados, int dia, int hora) {
```

```
    for (int i = 0; i < numero_solicitudes[dia][hora]; i++) {
      if (informacion_hilos[dia][hora][i].edo_solicitud == APLAZADA) {
        intercambiar_estados(&informacion_hilos[dia][hora][i],
                    &informacion_hilos[dia][hora+1][*recuperados]);
        (*recuperados)++;
      }
    }
  }
```

Utilidad del Proyecto

1. Aplicaciones Prácticas

- **Planificación de Recursos**: Modelado de distribución de recursos limitados
- **Análisis de Patrones de Consumo**: Estudio de comportamientos de usuarios
- **Optimización de Sistemas**: Identificación de cuellos de botella
- **Simulación de Escalabilidad**: Pruebas de carga y concurrencia

2. Valor Académico

- **Sistemas Operativos**: Implementación práctica de conceptos teóricos
- **Concurrencia**: Estudio de patrones de sincronización
- **Arquitectura de Software**: Diseño de sistemas distribuidos
- **Análisis de Algoritmos**: Optimización de recursos compartidos

3. Habilidades Técnicas Desarrolladas

- Programación concurrente en C
- Gestión de memoria compartida
- Sincronización entre procesos
- Diseño de arquitecturas distribuidas
- Análisis de rendimiento y eficiencia

Aprendizajes y Competencias Desarrolladas

1. Conceptos de Sistemas Operativos

Gestión de Procesos

- Creación y sincronización de procesos pesados
- Comunicación inter-proceso (IPC)
- Manejo de señales y terminación controlada

Sincronización y Concurrencia

- **Semáforos**: Coordinación de recursos compartidos
- **Mutex**: Exclusión mutua en secciones críticas
- **Reader-Writer Locks**: Acceso concurrente optimizado
- **Memoria Compartida**: Comunicación de alta velocidad

Gestión de Recursos

- Asignación y liberación de recursos limitados
- Detección y prevención de deadlocks
- Optimización de contención de recursos

2. Patrones de Diseño Implementados

Productor-Consumidor

```
// Auditor consume alertas generadas por nodos

sem_post(&shm->sem_auditor_listas);  // Productor

sem_wait(&shm->sem_auditor_listas);  // Consumidor
```

Readers-Writers

```
// Múltiples lectores concurrentes, escritor exclusivo

pthread_rwlock_rdlock(&shm->valvulas[i].rwlock_nodo);  // Lectura

pthread_rwlock_wrlock(&shm->valvulas[i].rwlock_nodo);  // Escritura
```

Barrier Synchronization

```
// Espera por todos los procesos antes de continuar

sem_wait(&shm->sem_nodo_residencial_listo_hora);

sem_wait(&shm->sem_nodo_industrial_listo_hora);
```

3. Competencias de Ingeniería de Software

Diseño de Arquitectura

- Modularidad y separación de responsabilidades
- Interfaces claras entre componentes

Depuración y Optimización

- Identificación de race conditions
- Análisis de rendimiento
- Optimización de secciones críticas

Documentación y Mantenimiento

- Código auto-documentado
- Comentarios descriptivos
- Estructura de proyecto clara

4. Habilidades Analíticas

Análisis de Complejidad

- Evaluación de complejidad temporal y espacial
- Identificación de cuellos de botella
- Optimización de algoritmos

Simulación y Modelado

- Modelado matemático de sistemas reales
- Generación de distribuciones estadísticas
- Validación de resultados

Métricas y Evaluación

- Recopilación de estadísticas
- Análisis de eficiencia
- Presentación de resultados

5. Lecciones Aprendidas

Sobre Concurrencia

- La importancia del orden de adquisición de locks
- Detección y prevención de deadlocks

Sobre Diseño de Sistemas

- La complejidad de la coordinación distribuida
- Importancia de la tolerancia a fallos
- Necesidad de logging y monitoreo

Sobre Desarrollo de Software

- Valor de la refactorización continua
- Importancia de pruebas exhaustivas
- Beneficios del diseño incremental

Conclusión

Eco Flow Simulation representa un proyecto integral que combina conceptos teóricos de sistemas operativos con implementación práctica de patrones avanzados de concurrencia. El sistema no solo demuestra dominio técnico en áreas críticas como sincronización de procesos y gestión de recursos compartidos, sino que también proporciona una base sólida para el desarrollo de sistemas distribuidos más complejos.

El proyecto sirve como excelente puente entre la teoría académica y la aplicación práctica, preparando a los desarrolladores para desafíos reales en el diseño y construcción de sistemas concurrentes y distribuidos a gran escala.